Control design — the circuit that drives the cart and finds its ends

The cart is moved by a stepper motor that reports nothing about where it is, so the circuit has to do three things the motor cannot: make the step pulses exactly, count every one of them, and say when the cart reaches an end of the rail. This file is the wiring for that circuit: an Arduino UNO R4 WiFi, a DM542 stepper driver, three small PNP transistors between them (S8550s, the ones on hand), two limit switches and a rocker that switches the Arduino's power. It serves both the rig in training and the jump-limit experiment (`jump-limit-experiment.md`, section 4, the harness's requirements).

The driver was a DRV8825 carrier until 2026-10-26. It could not carry the NEMA 23 motor's current (the user), so it is replaced by the DM542, which runs the motor at up to 4.2 A.

Status: built and working (2026-10-27). The DM542 wiring is on a breadboard (`control-breadboard.svg`), and `calibrate` homed the cart and centred it: span 59,828 pulses, 897.4 mm. The firmware's pulse timing now meets the DM542's (*What the firmware still needs*).

Assumptions

- **The DM542 follows Leadshine's DM542 manual** (*DM542 Fully Digital Stepper Drive Manual*, V1.0, 2012). Every pin name, switch table and timing below is from it unless another source is named; "the manual" means this one. DM542s from other makers copy the pinout and print the switch tables on the case: check the case of the driver in hand against *Setting the switches*.
- **The limit switches are lever microswitches** with three terminals: C (common), NO (normally open) and NC (normally closed).
- **The supply is 24 V.** The user's preference is 12 V, then 24 V, then 30 V (`motor-spec.md`), but the DM542 needs 20 V or more, so 12 V is out; 30 V fits the DM542 but not the Arduino's VIN (*Power*).

Open before wiring

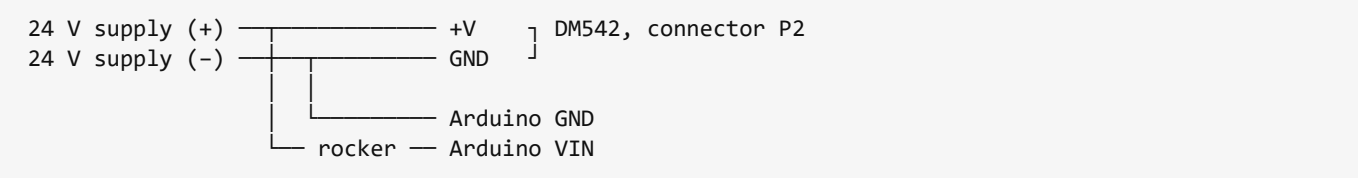
- **The rocker no longer turns the motor off.** With the DRV8825 the rocker put the driver to sleep, because its RESET and SLEEP pins hung on the Arduino's 5 V. The DM542 has no input like that: it is powered from the 24 V supply alone, so with the rocker off it stays on and keeps the motor energized and holding (*The power rocker*). A DM542 fault is cleared only by repowering the DM542, which the rocker does not do either.
- **The force the other docs use.** They assume 2.5 A, the DRV8825's maximum (`rig-drive-research.md`; `motor-spec.md`, 98 N at the belt). The DM542 is set to 2.69 A RMS (*Setting the switches*). By `motor-spec.md`'s own scaling, holding torque × current ÷ the motor's 4.0 A: 2.4 N·m × 2.69 ÷ 4.0 = 1.61 N·m, ÷ the pulley's 0.01528 m pitch radius = 106 N. Those two docs are not updated here; `jump-limit-experiment.md` is (2026-10-27: its ceiling is now about 6,200,000 pulses/s^2).

Terms

- **PUL, DIR, ENA.** The DM542's three command inputs, each a pair of pins (+ and −). Each pulse on PUL turns the motor one microstep; the level on DIR sets which way; ENA switches the driver's outputs off. They are the DM542's names for the DRV8825's STEP, DIR and ENABLE.

- **Optocoupler.** An LED and a light sensor in one package. Each DM542 input is one: current through the LED (in at +, out at −) is the signal, and the light carries it across, so the driver's power side and the Arduino share no wire (manual, section 4).
- **Lit.** An input whose LED has current through it.
- **Common-cathode wiring.** All three − pins go to ground, and each + pin is connected to 5 V by a switch to light its LED (manual, figure 4).
- **PNP transistor.** A switch worked by a small current: current drawn out of its *base* lets a larger current flow in at its *emitter* and out at its *collector*. Here each one is the switch that connects a + pin to 5 V. Its base is pulled low to switch it on, so a pin that drives it works upside down: low lights the input.
- **Microstep, pulses per revolution.** A fraction of the motor's 1.8° full step. 6,400 pulses per revolution is 1/32 of a full step per pulse, 15 µm of belt (`jump-limit-experiment.md`, section 2).
- **Peak and RMS current.** The DM542 sets the peak of the coil current; the RMS is the steady-current equivalent, 0.71 × the peak (manual, section 7, the current table). A motor's rated phase current compares with the RMS figure.
- **DIP switch.** The bank of eight small switches, SW1–SW8, on the DM542's side that sets its current and microstepping.
- **Pull-up.** A resistor that holds a line high when nothing else drives it.
- **IRQ.** An interrupt line: a pin change that makes the processor run a handler at once, instead of the firmware noticing on its next poll.
- **GPT.** The RA4M1's General PWM Timer, the hardware that makes the step pulses by itself (`rig-drive-research.md`, *The board*).

Power



- **+V takes 20–50 V** (manual, section 3, connector P2), and the manual suggests a supply of 20–45 V to leave room for the motor's back-EMF (section 6). 24 V is inside both.
- **No capacitor at the driver.** The manual asks for none; the 100 µF of the earlier design was the DRV8825 carrier's own requirement and goes with it.
- **The Arduino is powered on VIN from the same supply, through the rocker** (user, 2026-10-25; see *The power rocker*). VIN takes 6–24 V (*Sources*), so 24 V is its top: an adjustable supply is set no higher than 24.0 V.
- **The command signals need no ground shared with the DM542.** Each input's current comes from the Arduino's 5 V through its transistor and returns to the Arduino's GND; the optocoupler carries the signal across (manual, section 4). The supply's − still goes to both, because it powers the Arduino too.
- **The DM542 protects itself above 52 V** (manual, section 11).
- **Never connect or disconnect the motor while the DM542 is powered.** The manual: the motor current flows even at standstill, and pulling connector P2 makes a back-EMF surge that can damage the driver (section 8).

The driver's pins

Connector P1 (the command inputs):

DM542 pin	Connect to	Why
PUL+	**Q1's collector**; Q1's base from **D4**	D4 is P106, which carries GTIOC0B, the second output of GPT320, a 32-bit timer channel (RA4M1 user manual, the pin-function table), so the timer can make the pulses with no work per pulse. `unor4-hal`'s `pwm` drives GPT320 only on D5, its first output (`rig-drive-research.md`), so D4 is an addition to it, beside the runtime rate change STEP already needs. D4 low lights PUL. Firmware rule: D4 rests high; each pulse is low for at least 2.5 µs, then high for at least 2.5 µs (manual, section 10, t3 and t4), so 200 kHz at most (section 2).
PUL−	**GND**	Common-cathode wiring (manual, figure 4).
DIR+	**Q2's collector**; Q2's base from **D5**	Any digital pin. Firmware rule: change D5 only while D4 is high, and at least 5 µs before D4's next falling edge (manual, section 10, t2: DIR ahead of PUL's active edge by 5 µs). The manual's active edge is "rising" by default without saying which edge that is through a transistor; changing DIR only while D4 is high, 5 µs early, holds for either. This is how a reversal trial changes direction.
DIR−	**GND**	Common-cathode wiring.
ENA+	**Q3's collector**; Q3's base from **D6**	The firmware's own way to switch the motor off: D6 low lights ENA, which disables the driver and frees the motor; D6 high, or not driven, leaves it enabled (manual, section 3: the input is "usually left UNCONNECTED (ENABLED)"). Firmware rule: change D6 at least 5 µs before changing D5 (manual, section 10, t1).
ENA−	**GND**	Common-cathode wiring.

Connector P2 (power and motor):

DM542 pin	Connect to	Why
+V, GND	the supply	See *Power*.
A+, A−	motor black (A+), green (A−)	Coil A (`motor-spec.md`, *Wiring*, the maker's).
B+, B−	motor red (B+), blue (B−)	Coil B.

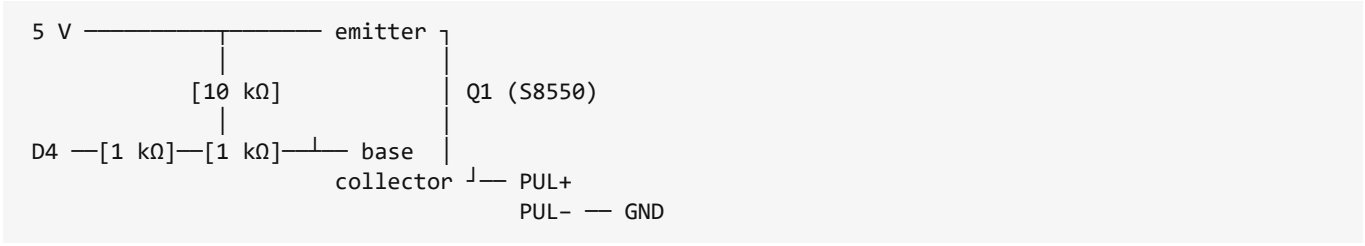
DIR's level stays undecided (`synthetic-arduino-plan.md`, *Not decided by the diagram*). No wiring depends on it: swapping one coil's two wires reverses the motor, and so does the firmware's sign.

The command wires run at least 10 cm from the motor's four leads (manual, section 8), which recommends twisted, shielded cable for them.

There is no fault line. The DM542 shows an over-current or over-voltage fault only by blinking its red LED and freeing the motor shaft, and clears it only when repowered (manual, section 11). The Arduino cannot see a fault; it shows up as a cart that is not where the pulse count says, found at the next limit-switch trip. D8, the DRV8825's FAULT input, is unused.

Why the transistors

The Arduino's pins cannot light the inputs themselves. Each input draws 7–16 mA, 10 mA typical (manual, section 2). D4, D5 and D6 are the RA4M1's P106, P107 and P111, which can give out and take in 4 mA at most (RA4M1 datasheet, Table 2.6, ports P100 to P115, low drive). Low drive is the pins' setting: the drive-capability bit (DSCR in PmnPFS) is 0, low drive, after reset (RA4M1 user manual, 19.2.5), and `unor4-hal` never sets it. Middle drive would allow 8 mA taken in. So each pin drives a small PNP transistor, and the transistor carries the input's current from 5 V: the manual's PNP connection (figure 4).



Q2 on D5 and DIR, and Q3 on D6 and ENA, are the same. D4, D5 and D6 sit side by side on the header, in the order of Q1, Q2 and Q3 on the breadboard.

The breadboard layout is `control-breadboard.svg`, hole by hole, with each resistor's value printed beside it. Each stage's path is pin → 1 kΩ → 1 kΩ → base → 10 kΩ → + rail. Two faults found in the first build (2026-10-27): the 1 kΩ and 10 kΩ swapped on every stage (their colour codes differ in one band), and a breadboard whose power rails are broken halfway along. On such a board, jumper the two halves of each rail.

- **The input current, by hand:** 5 V, less about 0.2 V across the switched-on transistor and about 1.2 V across the input's LED, over the 270 Ω the manual draws inside each input (figure 10): (5 − 0.2 − 1.2) ÷ 270 ≈ 13 mA, inside 7–16 mA. The 0.2 V and 1.2 V are typical figures, from no source. The manual's 1 kΩ and 2 kΩ series resistors are for 12 V and 24 V signals (section 2); at 5 V none is needed.
- **The base resistor, 2 kΩ as two 1 kΩ in series:** with the pin low, the pin takes in (5 − 0.7) ÷ 2,000 ≈ 2.2 mA, inside its 4 mA. A single 1 kΩ would draw 4.3 mA, over it. That base current switches 13 mA with a wide margin: 13 ÷ 2.2 ≈ 6, against an S8550's gain of 85 at the lowest grade (B) and 160–300 at grade D, the "D" on the parts in hand (the S8550 datasheets).
- **Pin high switches it off.** The RA4M1 runs on the board's 5 V ("The RA4M1 and the GPIOs of this board operates on 5 V", Arduino's UNO R4 WiFi page), the same 5 V as the emitters, so a high pin holds the base at the emitter's voltage and no base current flows.
- **The 10 kΩ pull-up** holds the base at 5 V while its pin is not driven (at boot, at rocker-on, during a reset), so the transistor is off: no stray pulse reaches PUL and ENA stays enabled. It takes 0.07 mA of the base current when the pin is low.
- **The firmware sets each pin high before driving it.** `unor4-hal`'s `into_output_in_state(PinState::High)` writes the level before it turns the pin into an output, so going from undriven to driven lights nothing.
- **Pin low means lit.** The firmware works the three pins upside down: D4 rests high and pulses low, D6 low disables the driver.
- **The S8550 is rated −25 V and −500 mA** (the S8550 datasheets), far above 5 V and 13 mA. Its leg order differs by maker (emitter, base, collector on LGE's sheet; collector, base, emitter on Blue Rocket's), so check the parts in hand: on a multimeter's diode setting the base is the leg that reads as a diode to each of the other two.
- **The 1 kΩ and 10 kΩ are the parts on hand (user, 2026-10-27), and the 2 kΩ is this design's choice, from no source**; any base resistor from about 1.1 kΩ (4 mA) to 10 kΩ works, and any small PNP with a gain of 85 or more serves.
- **The 10 kΩ must be the pull-up and the 1 kΩ the base resistors.** Swapped, the 1 kΩ sits across base and emitter and takes almost all the current the 11 kΩ chain can pull: the base settles at 5 × 11 ÷ 12 ≈ 4.6 V, 0.4 V below the emitter, and the transistor barely switches on (measured 2026-10-27: base 4.6 V, collector 0.43 V).

Setting the switches

Switch	Setting	Means

SW1, SW2, SW3	on, off, off	3.76 A peak, 2.69 A RMS (manual, section 7, *Dynamic Current Setting*)
SW4	on	the standstill current stays at the full current; off would drop it to half a second after the last pulse (section 7, *Standstill Current Setting*)
SW5, SW6, SW7, SW8	off, on, off, on	6,400 pulses per revolution, 1/32 microstep (section 7, *Microstep Resolution Selection*)

- **The pulse size is unchanged.** 6,400 pulses per revolution is the DRV8825 design's 1/32, so one pulse is still 15 µm of belt.
- **The current.** For a 4-lead motor the manual sets the peak at the rated phase current × 1.4 (section 5): 4.0 A × 1.4 = 5.6 A for the 23HS32-4004S, above the DM542's top setting of 4.20 A. So the motor runs below its rating at any setting. 3.76 A, one step below the top, is the setting chosen on 2026-10-26; its reason was not recorded.
- **SW4 on** keeps the full current at standstill; the manual recommends the half-current mode, to cut heating (section 2, *Elimination of Heat*).
- **Auto-tuning.** The first time the DM542 is powered with the motor connected, and again after changing the motor or the supply, flip SW4 twice within one second with no pulses arriving: on, off, on, so it ends on (manual, section 7, *Auto Tuning by SW4*). The DM542 measures the motor and sets its current control; the shaft vibrates a little while it does.
- **The two jumpers inside, J1 and J3, stay open:** pulse-and-direction mode, rising edge active, the default (manual, section 3).

The limit switches

The switch at the first end homing finds is the rig's position reference: the jump-limit experiment notes the pulse count at the instant it trips (`jump-limit-experiment.md`, section 4). The switches are sensed, never switched into power (user, 2026-10-25): each is only an input to the Arduino, and nothing in the circuit cuts the driver or the motor when one trips. What a trip means (note the count, stop sending pulses) is the firmware's. `calibrate` treats either switch as an end of the rail and names the pin that tripped at each end (user, 2026-10-27), so its homing does not depend on which switch is wired to which pin; both ends tripping one pin halts it as a wiring fault.

```
D2 ── switch NC   (one switch; the other is the same on D3)
GND ── switch C
```

- **The NC contact is used.** Away from the switch the contact is closed and the pin reads low; the cart pressing the switch opens it, and the pin reads high. A broken or unplugged wire also reads high, so the firmware sees a wiring fault as a trip and stops sending pulses, instead of driving the cart into the end.
- **D2 and D3 are the pins because they have IRQs:** D2 is P104 (IRQ1), D3 is P105 (IRQ0) (RA4M1 user manual; the D-to-P mapping is `unor4-hal`'s `gpio/pins.rs`). The reference count is taken in the interrupt handler, not by polling.
- **No resistor, no capacitor, no 5 V.** The pull-up that makes the pin read high when the contact opens is the board's own, switched on by the firmware (`calibrate/src/axis.rs`, `into_input_pullup`). The switches worked wired this way, with no added parts (user, 2026-10-27).
- Each switch's two wires are twisted together and kept apart from the motor's four leads.

The power rocker

The rocker switches the Arduino's power on VIN (user, 2026-10-25). It does not switch the DM542, which is powered from the supply directly.

```
24 V supply (+) ── rocker ── Arduino VIN
24 V supply (–) ──────────── Arduino GND
```

- **VIN takes 6–24 V** (Arduino's UNO R4 WiFi page, https://docs.arduino.cc/hardware/uno-r4-wifi/, read 2026-10-25). The 24 V supply fits at the top of that range; the 30 V supply must not go to VIN.
- **Rocker off:** the Arduino is off and its 5 V falls to 0, so no input can be lit: no pulses, and ENA unlit means enabled. The DM542 stays powered and keeps the motor energized, holding the cart still (at full current, with SW4 on). The rocker no longer frees the motor (*Open before wiring*).
- **Rocker on:** the Arduino boots. Its pins are undriven until the firmware sets them high, and the pull-ups keep the transistors off meanwhile, so nothing moves until the firmware sends pulses.
- **USB bypasses the rocker.** With the USB cable plugged in, the board takes its power from USB and the rocker no longer turns it off. The Arduino page does not describe the USB/VIN power path; this is from general knowledge of the board, not a source read for this design.
- **The rocker's rating:** the supply voltage, and at least 1 A. The Arduino page gives no current figure for the board; 1 A is this design's margin, from no source.

Switching it off loses the pulse count. The motor holds the cart, but the cart can still be pushed past the motor's holding torque, and every power-up starts the firmware fresh, with no count. So it starts not homed and homes (both ends, then the centre: `synthetic-loop-dia.md`, *Numbers*) before accepting any command. A jump-limit trial interrupted by the rocker is void.

The pin map

Arduino	Role
D2	left limit switch (IRQ1)
D3	right limit switch (IRQ0)
D4	PUL (GPT320's second output), through Q1 (rests high, pulses low)
D5	DIR, through Q2 (low lights it)
D6	ENA, through Q3 (low disables the driver)
D7	unused (ENA's pin before 2026-10-27)
D8	unused (the DRV8825's FAULT)
VIN	the supply's +, through the rocker
5 V	Q1–Q3's emitters and pull-ups
GND	the supply's −, PUL−, DIR−, ENA−, the switches' C terminals

What the firmware still needs

The circuit does not depend on it, but the rig's training firmware cannot run until it has a STEP output on the board's timer whose rate changes while running, and a pulse count; `unor4-hal` has neither yet (`rig-drive-research.md`, *The user's `unor4-hal`*).

The `calibrate` firmware (`calibrate/src/axis.rs`) does not wait for that: it makes every pulse itself, in software, and counts it. It drives the pins the PNP way: D4 rests high and pulses low, D6 low disables the driver. Its timing is set in `calibrate.toml` and checked against the DM542's at build time: each pulse 5 µs on (`pulse_low_ns`, the DM542 needs 2.5 µs on and 2.5 µs off), DIR set 5 µs before it (`dir_setup_ns`, the DM542's t2). It also runs the jump-limit experiment (`jump-limit-experiment.md`) from its page, timing a trial's pulses on the processor's cycle counter so each jump lands where the plan puts it. It halts when D8 reads low; D8 is unconnected and its pull-up holds it high, so that check never trips.

Sources

- Leadshine, *DM542 Fully Digital Stepper Drive Manual*, V1.0, 2012 (copy read 2026-10-27 from https://forum.linuxcnc.org/media/kunena/attachments/25011/dm542_2019-12-07.pdf): the supply range and the 52 V protection, the input current, pulse width and 200 kHz (section 2); the pins, ENA's sense, the jumpers (section 3); the optocoupler inputs and the PNP connection (section 4, figure 4); the 1.4 × phase-current rule (section 5); the supply advice (section 6); the switch tables and auto-tuning (section 7); the wiring notes and the hot-plug warning (section 8); the 270 Ω inside each input (figure 10); the timing t1–t4 (section 10); the fault LED (section 11).
- Arduino UNO R4 WiFi — https://docs.arduino.cc/hardware/uno-r4-wifi/ (read 2026-10-25): VIN's 6–24 V; the RA4M1 and its pins on 5 V (read 2026-10-27).
- RA4M1 Datasheet rev 1.10, Table 2.6: ports P100 to P115 give out and take in 4.0 mA at most at low drive; at middle drive they give out 4.0 mA and take in 8.0 mA — `D:\git\unor4-hal\Research\Renesas RA4M1 Datasheet - 1.10.md`.
- RA4M1 User's Manual rev 1.10, 19.2.5 (PmnPFS): DSCR is 0, low drive, after reset. `unor4-hal`'s `gpio` never writes it.
- S8550 datasheets: LGE (https://wmsc.lcsc.com/wmsc/upload/file/pdf/v2/lcsc/2008031905_LGE-S8550_C717454.pdf) and Blue Rocket (https://wmsc.lcsc.com/wmsc/upload/file/pdf/v2/lcsc/2108072030_Foshan-Blue-Rocket-Elec-8550-C_C2844066.pdf), found 2026-10-27: −25 V, −500 mA, the gain grades, the leg orders.
- `unor4-hal` `crates/unor4-hal/src/gpio/pin.rs`, `into_output_in_state`: the level is written before the pin becomes an output.
- RA4M1 User's Manual rev 1.10, the pin-function table: P104 IRQ1, P105 IRQ0, P106 GTIOC0B — `D:\git\unor4-hal\Research\Renesas RA4M1 User Manual - 1.10.md`.
- `unor4-hal` `crates/unor4-hal/src/gpio/pins.rs`: the WiFi board's D-to-P pin mapping; the GPT channel per pin, via `rig-drive-research.md`.
- `motor-spec.md`: the motor's wire colours, coils and 4.0 A rating; the torque scaling; the supply preference.
- No source: the 2 kΩ base resistor, the LED and transistor voltage drops, the NC wiring, the rocker's 1 A rating, USB bypassing the rocker. Each has its reason where it appears.